

DFD LEVEL 0 & 1 ARCHITECTURE GUIDE

AeroBook Flight Platform

Complete Data Flow Diagram & Pipeline Specification

End-to-end transformation pipelines, isolated microservice database boundaries, asynchronous messaging topologies, and gate clearance flows.

DATA ARCHITECTURE SPECIFICATIONS

Notation Standard:

Gane & Sarson / Yourdon

Core Processes:

7 Transformation Engines

Visual Theme:

Black Canvas & White Marker

Database Isolation:

Database-per-Service (MySQL 8)

External Entities:

5 External Source/Sinks

Event Broker:

RabbitMQ (AMQP 0-9-1)

EXECUTIVE ARCHITECTURAL SUMMARY

AeroBook is a high-availability, mission-critical flight reservation ecosystem engineered using microservices principles. This document details the complete Data Flow Architecture of the platform across both Context (Level 0) and Decomposition (Level 1) perspectives.

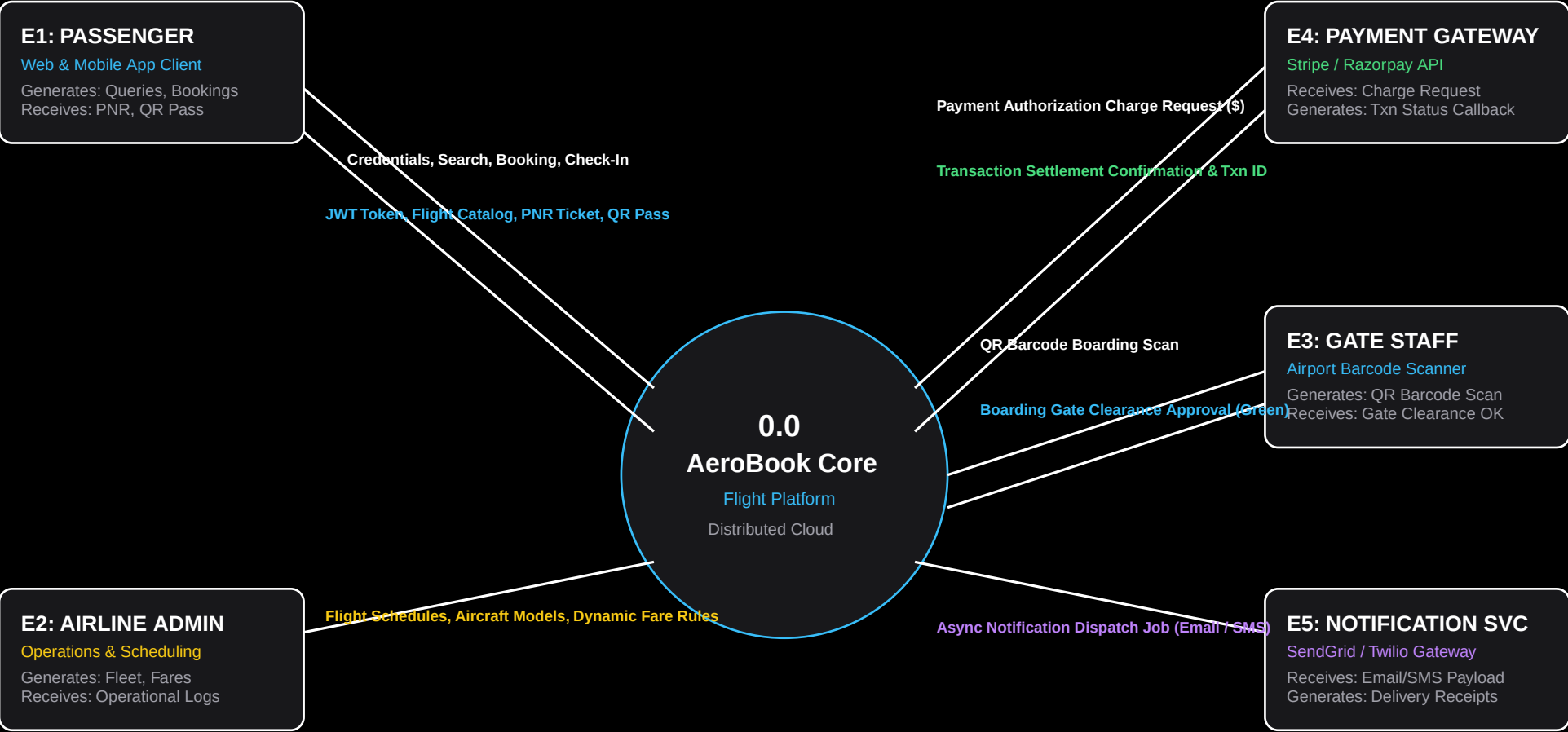
The data topology enforces absolute Database-per-Service encapsulation: each domain service manages an isolated MySQL schema with zero direct cross-database queries or shared memory locks. Cross-domain coordination is performed via synchronous REST contracts over HTTPS for instant user feedback (Seat locking, Fare computation, Payment gateway handshakes) and asynchronous RabbitMQ event publishing for non-blocking workloads (Itinerary generation, SMS notifications, and audit persistence).

DOCUMENT OUTLINE & ROADMAP

Page 2	System DFD Level 0 Context Diagram	- System boundary, external actors (E1-E5), and macro-level data flows.
Page 3	System DFD Level 1 Subsystem Flow Architecture	- Embedded full-resolution landscape vector blueprint of all 7 processes and 7 data stores.
Page 4	Process Specifications: P1.0 to P4.0	- Auth, Profile, Flight Discovery, and Dynamic Fare engine data pipelines.
Page 5	Process Specifications: P5.0 to P7.0	- Transactional Booking Saga, Web Check-In, QR Generation, and RabbitMQ worker.
Page 6	Data Store Dictionary & Database Isolation	- MySQL schemas (D1 to D6) and RabbitMQ queue topology (D7).
Page 7	End-to-End Execution Cycles & Payloads	- Step-by-step trace of Search-to-Booking, Check-in, and Async Notification pipelines.
Page 8	Security Guarantees & Technical Defense FAQ	- PCI-DSS tokenization, BCrypt, eventual consistency, and viva examination defense.

DFD Level 0: System Context Diagram

Macro-level data boundaries showing external entities interacting with the AeroBook black box core.



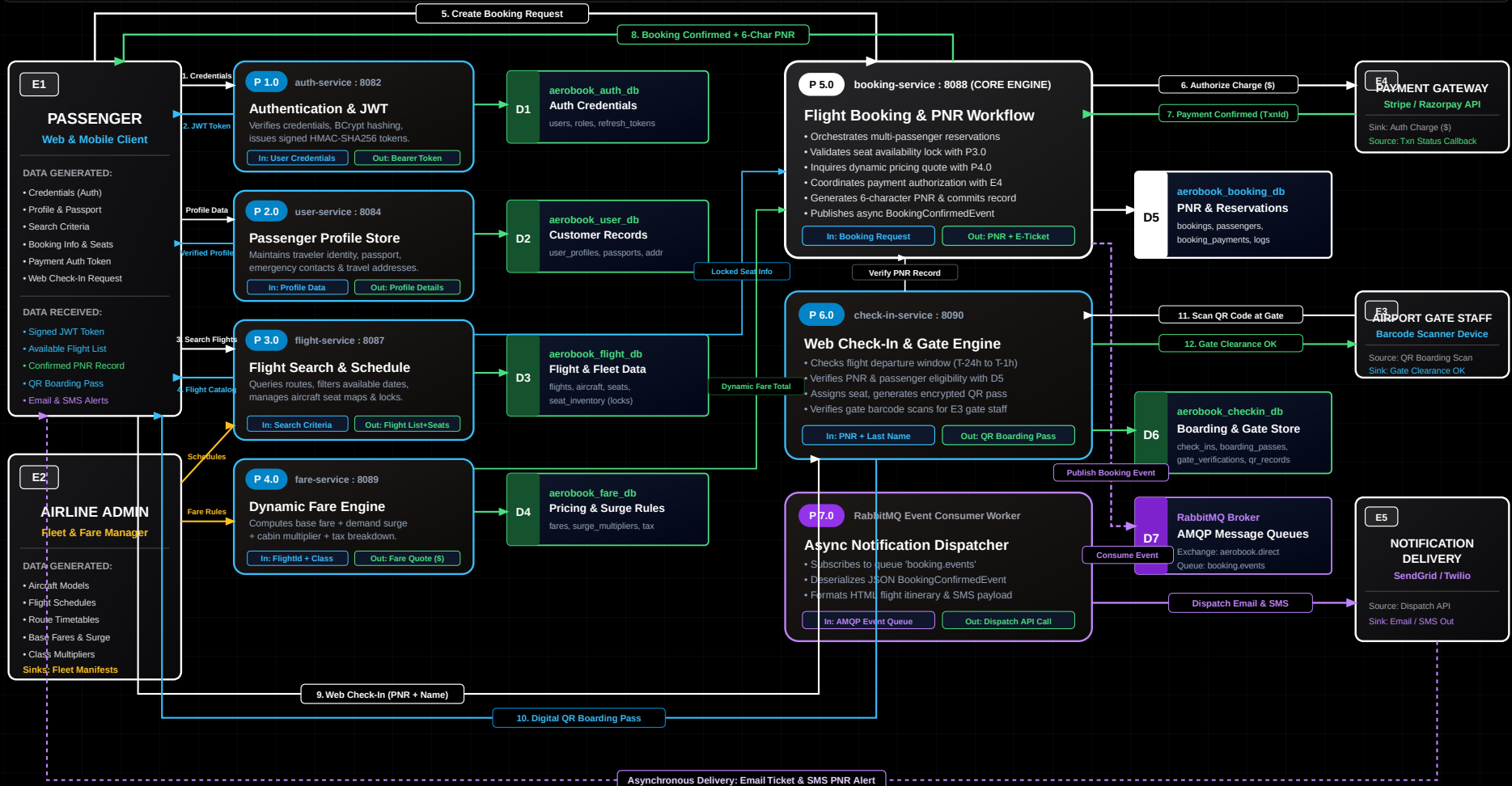
NOTATION GUIDE: External Entity [E#] (Source/Sink)

Process [P#] (Transformation Node)

Data Store [D#] (Isolated DB / Broker)

Synchronous REST Data Flow

Async Event Notification (RabbitMQ)



AEROBOOK DATA FLOW GOVERNANCE & ARCHITECTURAL GUARANTEES

1. Database-Per-Service Isolation (D1 - D6)

Each microservice encapsulates its own MySQL schema (aerobook_auth_db, aerobook_flight_db, etc.). Direct foreign keys across databases are strictly prohibited; entity correlation occurs via immutable UUIDs and PNRs. Prevents tight coupling, deadlock cascades, and allows independent scaling and horizontal database replication.

2. Transaction Boundary & Payment Security (E4, P5.0)

Zero plain-text credit card storage. Card tokens are authorized exclusively through third-party gateways (E4). Booking saga coordinates two-phase seat lock verification before committing the reservation in aerobook_booking_db. If payment fails or times out, the transient seat hold is immediately relinquished back into the flight inventory.

3. Eventual Consistency & AMQP Messaging (D7, P7.0)

High-throughput, asynchronous decoupling via RabbitMQ message broker with guaranteed delivery. Booking confirmation emails and SMS alerts are processed out-of-band by asynchronous notification consumers. Dead-Letter Exchanges (DLX) and retry backoff policies ensure zero notification drop during high-load surges.

Process Specifications: P1.0 to P4.0

Input/Output Pipelines, Algorithmic Logic & Microservice Contracts

P 1.0

Authentication, Access Control & Token Issuance

auth-service (Port 8082)

Acts as the central identity provider. Validates credentials, manages role authority hierarchies, and issues signed cryptographic JWT tokens.

Input Data: Login credentials (username, password), Registration payload, Refresh tokens.

Output Data: HMAC-SHA256 Bearer JWT token, Role authorities ([ROLE_PASSENGER, ROLE_ADMIN]), Expiration claim.

Encapsulated DB: aerobook_auth_db (users, roles, user_roles, refresh_tokens)

Core Algorithm: BCrypt salt-hash evaluation (strength 10). Generates stateless JWT tokens carrying username, roles, and 24-hour expiration window.

P 2.0

Passenger Profile & Customer Identity Records

user-service (Port 8084)

Maintains traveler contact details, passport data, date of birth, frequent flyer mileage, and emergency contact registries.

Input Data: Profile update requests, Government passport records, Residential address updates.

Output Data: Verified customer profile entity, Identity audit confirmation, Profile completion score.

Encapsulated DB: aerobook_user_db (user_profiles, passports, addresses)

Core Algorithm: Validates international passport formatting and expiration dates. Implements GDPR compliance data rectification.

P 3.0

Flight Search & Fleet Inventory Scheduling

flight-service (Port 8087)

Queries real-time flight schedules, calculates available seat matrices by cabin class, and manages atomic temporary seat locks.

Input Data: Flight search filters (Origin, Destination, Date, Cabin), Admin fleet timetables, Seat lock requests.

Output Data: Matching flight catalog cards, Real-time cabin seat occupancy map, 10-minute seat hold tokens.

Encapsulated DB: aerobook_flight_db (flights, aircraft, seats, seat_inventories)

Core Algorithm: High-speed index scan on (origin_airport, destination_airport, departure_time). Atomic seat lock prevents double-booking.

P 4.0

Dynamic Fare & Surge Pricing Calculation Engine

fare-service (Port 8089)

Computes real-time dynamic pricing based on distance base fares, cabin multipliers, seasonal demand surge, and airport taxes.

Input Data: Flight ID, Cabin Class, Booking timestamp, Demand occupancy ratio.

Output Data: Itemized fare quote (Base fare, Demand surge, Fuel surcharge, Airport departure taxes, Final Total).

Encapsulated DB: aerobook_fare_db (fares, surge_multipliers, class_pricing_rules)

Core Algorithm: $FinalPrice = BaseFare * CabinMultiplier * (1 + DemandSurgeRate) + AirportTaxes$. Recalculated at quote generation.

Process Specifications: P5.0 to P7.0

Transactional Saga Coordinator, Gate Verification & Event Consumer

P 5.0

Flight Booking & PNR Orchestration Engine

booking-service (Port 8088) - PLATFORM CORE

The mission-critical business coordinator. Coordinates multi-passenger booking, coordinates seat locks with P3.0, dynamic quotes with P4.0, payment authorization with E4, generates immutable 6-character PNRs, and publishes AMQP events.

Input Data: Booking request (flight ID, passenger manifest, seat allocations, payment method token).

Output Data: Confirmed PNR record, E-ticket document, BookingConfirmedEvent payload published to RabbitMQ queue.

Encapsulated DB: aerobook_booking_db (bookings, passengers, booking_payments, booking_audit_logs)

Core Algorithm: Booking Saga Coordinator: Atomic validation of seat hold -> E4 Payment authorization -> Commit PNR record -> Publish RabbitMQ AMQP event. On failure, seat hold is released immediately.

P 6.0

Airport Web Check-In & Gate Clearance Engine

check-in-service (Port 8090)

Manages airport departure eligibility, seat confirmation, cryptographic 2D QR boarding pass generation, and real-time barcode validation for gate staff embarkation scanners.

Input Data: Passenger check-in queries (PNR + Last Name), Gate barcode scanner laser reads from E3.

Output Data: Digital boarding pass with QR code, Boarding sequence number, Gate clearance approval (Green audio/visual).

Encapsulated DB: aerobook_checkin_db (check_ins, boarding_passes, gate_verifications)

Core Algorithm: Departure Window Enforcer: Only permits check-in between T-24h and T-1h before departure. QR payload incorporates PNR, seat, and HMAC-SHA256 signature to prevent boarding pass forgery.

P 7.0

Asynchronous Event Broker & Notification Dispatcher

event-broker (Port 5672) - RabbitMQ Consumer & Notification Broker

Consumes business events from RabbitMQ message queues out-of-band, formats HTML electronic itineraries, and handles third-party delivery to SMTP and SMS gateways with retry resilience.

Input Data: AMQP message payload (BookingConfirmedEvent, CheckInCompletedEvent) from queue booking.events.

Output Data: Dispatch API calls to SendGrid (Email ticket) and Twilio (SMS PNR alert).

Encapsulated DB: RabbitMQ Durable Queues (booking.events, booking.dlg) & In-memory consumer worker

Core Algorithm: At-Least-Once Delivery: Worker consumes message, processes notification dispatch, and sends basicAck only upon successful delivery. On unrecoverable error, message is dead-lettered.

Data Store Dictionary & Isolation Topology

Database-Per-Service Specifications, Storage Technologies & Table Schemas

STORE	DATABASE NAME	TECH	PRIMARY TABLES / QUEUES	KEYS & IDENTIFIERS
D1	aerobook_auth_db	MySQL 8.0	users, roles, user_roles, refresh_tokens Stores encrypted passwords (BCrypt), role authorizations, and token revocation registries.	PK: id, UK: username, email
D2	aerobook_user_db	MySQL 8.0	user_profiles, passports, addresses Encapsulates customer identity, international passport details, travel preferences, and contact info.	PK: id, UK: user_id, passport_number
D3	aerobook_flight_db	MySQL 8.0	flights, aircraft, seats, seat_inventories Flight schedules, aircraft cabins, seat status (AVAILABLE, HELD, BOOKED), and departure gates.	PK: id, UK: flight_number + departure_time
D4	aerobook_fare_db	MySQL 8.0	fares, surge_multipliers, class_pricing_rules Base distance pricing tables, load factor surge multipliers, and seasonal tax formulas.	PK: id, UK: flight_id + cabin_class
D5	aerobook_booking_db	MySQL 8.0	bookings, passengers, booking_payments, audit_logs Core ACID reservation repository. Tracks passenger tickets, booking state, and payment references.	PK: id, UK: pnr (6 chars), txn_id
D6	aerobook_checkin_db	MySQL 8.0	check_ins, boarding_passes, gate_verifications Check-in timestamps, assigned boarding sequences, QR verification signatures, and embarkation logs.	PK: id, UK: boarding_pass_id, pnr + seat
D7	RabbitMQ Message Broker	AMQP 0-9-1	Exchanges: aerobook.direct Queues: booking.events Persistent, durable event broker decoupling booking completion from notification delivery.	Routing Key: booking.created

WHY DATABASE-PER-SERVICE IS CRITICAL FOR AEROBOOK

- Deadlock Elimination: A traffic surge in flight seat searches never blocks booking checkout tables.
- Failure Domain Containment: An outage in user profile updates does not disrupt airport check-in scanners.
- Independent Scalability: flight_db can replicate to read-replicas during holiday booking surges.
- Schema Autonomy: Fare calculation logic can upgrade without triggering migration downtime on booking databases.

Execution Cycles & Data Flow Payloads

JSON Data Structures Exchanged Across Microservice Boundaries

1. Booking Creation Request Payload

POST /api/bookings (E1 -> P5.0)

```
{  "flightId": 1087,  "cabinClass": "ECONOMY",  "contactEmail": "traveler@aerobook.com",  "paymentMethodToken": "tok_visa_4242",  "passengers": [    {      "firstName": "Alex",      "lastName": "Vance",      "passportNumber": "P9842103",      "seatNumber": "14B"    }  ]}
```

2. Payment Gateway Authorization

Base64 2D Barcode (P6.0 -> E1 & E3)

3. AMQP Event Published to RabbitMQ

Exchange: aerobook.direct (P5.0 -> D7 -> P7.0)

```
{  "eventType": "BookingConfirmedEvent",  "eventId": "evt_491823019",  "pnr": "AB74K2",  "flightNumber": "AB-402",  "origin": "JFK",  "destination": "LHR",  "departureTime": "2026-10-15T08:00:00Z",  "passengerEmail": "traveler@aerobook.com",  "passengerName": "Alex Vance",  "seatNumber": "14B",  "totalAmount": 349.50}
```

4. QR Boarding Pass Payload

TRANSACTION LIFECYCLE SUMMARY

- Phase 1: Synchronous Booking Handshake: Passenger submits traveler details. Booking service executes atomic seat check and calls Payment Gateway. Once authorized, PNR record is committed into aerobook_booking_db.
- Phase 2: Asynchronous Out-of-Band Delivery: Booking service publishes BookingConfirmedEvent to RabbitMQ in under 5ms. Notification consumer picks up event in background and dispatches SendGrid email.
- Phase 3: Airport Web Check-In & Embarkation: At T-24h, passenger checks in. QR pass is issued. At airport, Gate Staff scans barcode. Check-in service validates cryptographic signature and clears passenger to board.

Security Guarantees & Viva Defense FAQ

Essential Questions, Architecture Defense & Security Principles

Q1: Why are credit card numbers (PAN) never stored in the AeroBook database?

AeroBook strictly enforces PCI-DSS Level 1 Scope Minimization. Credit card data is tokenized on the client side directly with Stripe/Razorpay. AeroBook only receives and stores opaque, single-use tokens and transaction IDs, ensuring zero cardholder data breach exposure.

Q2: How does the system handle a network failure between Booking Service and RabbitMQ?

The Booking Service uses Spring AMQP with publisher confirms enabled. If the RabbitMQ broker cannot be reached or rejects the message, the event is saved to a local outbox table inside the booking database transaction. A scheduled worker replays failed outbox events.

Q3: Why is Check-In separated into its own service instead of being inside Booking Service?

Airport check-in has drastically different scalability characteristics and uptime requirements. On high-travel days, airport gate scanners must operate with zero latency and 99.999% availability even if the main booking or flight search services are experiencing maintenance.

Q4: How does AeroBook prevent double-booking when multiple users attempt to book the same seat?

Flight Service maintains an atomic seat status lock table with timestamped hold expirations. When a user selects a seat, an atomic database row lock (SELECT ... FOR UPDATE) sets status to HELD for 10 minutes. Subsequent concurrent requests for that seat are rejected.

CONCLUSION: PRODUCTION-READY DATA PIPELINE

The AeroBook Data Flow Architecture successfully decouples high-throughput read operations (Flight search, Dynamic fare calculation) from mission-critical transactional writes (PNR booking, Seat inventory locking) and airport gate security workflows. Through clear external entity boundaries, strict database isolation, and AMQP event streaming, the platform achieves high availability, fault isolation, and enterprise-grade reliability.

